

計算機科学基礎実験 レポート 4

氏名：加藤悠菜 / 学籍番号：6325040

2026 年 6 月 2 日

1 実験結果のまとめ

1.1 要素数による比較回数の変化

表 1 実験 1

ソート名	要素数 10	要素数 50	要素数 100
交換ソート (exchange_sort_c)	81 回	1,715 回	8,019 回
選択ソート (selection_sort_c)	45 回	1,225 回	4,950 回
挿入ソート (insection_sort_c)	32 回	618 回	2,601 回
マージソート (merge_sort)	25 回	220 回	541 回
クイックソート (quick_sort_c)	20 回	212 回	509 回

1.2 データの初期状態による変化

表 2 実験 2

ソート名	整列済み	逆順	ランダム
交換ソート (exchange_sort_c)	19 回	380 回	266 回
選択ソート (selection_sort_c)	190 回	190 回	190 回
挿入ソート (insection_sort_c)	19 回	190 回	105 回
マージソート (merge_sort)	69 回	69 回	64 回
クイックソート (quick_sort_c)	190 回	190 回	62 回

2 データの初期状態とアルゴリズムの特性

「交換ソート」「選択ソート」「挿入ソート」について、データの初期状態が比較回数に与える影響をプログラムの構造面から考察する。

■**交換ソート (exchange_sort_c)** 実験2のデータより、交換ソートの比較回数は「整列済み：19回」「ランダム：266回」「逆順：380回」と変動した。この原因は、関数 `exchange_sort_c` における、補助関数 `exchange_pass_c` から返される真偽判定の `judge`（要素の交換が発生したか）による条件分岐構造にある。

データが「整列済み」の場合、1周目の `exchange_pass_c` ですべての隣り合う要素の比較 ($N - 1 = 19$ 回)が行われるが、一度も `if h1 > h2` が成立しないため `judge` は `false` となる。これにより `exchange_sort_c` の再帰が即座に打ち切れ、最小回数である19回で終了する。一方、「逆順」データの場合は、各周回で必ず交換が発生して `judge` が `true` となり、完全に整列するまで最大数のパスを繰り返すことになる。そのため、380回という非常に多い比較回数を記録した。

■**選択ソート (selection_sort_c)** 実験2のデータより、選択ソートの比較回数は「整列済み」「逆順」「ランダム」のいずれの初期状態においても、一貫して「190回」から一切変動しなかった。この原因は、補助関数 `select_min_c` の無条件な走査構造にある。

プログラムの構造を見ると、`select_min_c` はリストの先頭要素 `h` と残りのリストの最小値 `min_val` を比較する前に、必ず `let (min_val, rest, count) = select_min_c t in` によってリストの末尾に到達するまで完全に再帰を繰り返す。データの並び順がどうであれ、リストの走査を途中で打ち切る仕組みがコード内に存在しないため、どのような初期状態であっても必ず最大スキャンが実行される。結果として、一律で比較が190回行われる特徴がある。

■**挿入ソート (insertion_sort_c)** 実験2のデータより、挿入ソートの比較回数は「整列済み：19回」「ランダム：105回」「逆順：190回」と、交換ソート同様に変動があった。この原因は、補助関数 `insert_c` の条件分岐による早期打ち切り構造にある。

新しく要素 `a` をソート済みリストに挿入する際、データが「整列済み」であれば、`if a <= h` という条件が常に最初の1回目の比較で成立する。そのため、`else` 以下の再帰呼び出しがすべて打ち切れ、各ステップで1回しか比較が発生しない。要素数 $N = 20$ に対し最良のケースとして $N - 1 = 19$ 回となる。しかし、「逆順」データの場合は、常に手札の中で最も小さい要素を最奥部まで挿入し続けることになるため、`if a <= h` が決して成立しない。結果として処理の打ち切りが発生せず、毎回リストの末尾まで再帰呼び出しを繰り返すため、選択ソートと同じ190回となった。「ランダム」では、平均的にリストの中間付近で打ち切られるため、理論上の平均値に近い105回となった。

3 分割統治法の効率性

実験 1 において、要素数が 10 から 100 へと増加した際、単純ソートと分割統治ソートの間には、比較回数の増え方に大きく違いが見られた。

3.1 実験データから読み取れる事実

要素数が要素数が 10 から 100(**10 倍**) になったときの、各ソートの比較回数の推移および増加倍率を以下に示す。

- 交換ソート：81 回 → 8,019 回（約 99.0 倍に増加）
- 選択ソート：45 回 → 4,950 回（110.0 倍に増加）
- 挿入ソート：32 回 → 2,601 回（約 81.3 倍に増加）
- マージソート：25 回 → 541 回（約 21.6 倍に増加）
- クイックソート：20 回 → 509 回（約 25.5 倍に増加）

単純ソートは、要素数が 10 倍になると比較回数が**約 80 倍～110 倍**へと大きく増加していることが分かる。一方で、分割統治ソートは、要素数が 10 倍になっても比較回数は **20 倍強**という極めて緩やかな増加である。要素数 100 の場合、最も比較回数が多い交換ソート（8,019 回）と最も少ないクイックソート（509 回）の間には、すでに約 15.7 倍もの圧倒的な速度差が生じている。

3.2 分割統治法が大量のデータを扱う上で有利な理由

分割統治法が大量のデータを処理する際に圧倒的に有利である理由は、配列を「要素数 1」になるまで半分ずつに分割し、**小さい要素同士でのみ比較・統合を行う**という構造を持つためである。単純ソート（交換・選択・挿入）は、データを 1 つの巨大な塊のまま扱い、全要素に対して愚直に何度も走査を繰り返す。そのため、要素数 N に対して計算量が $O(N^2)$ となり、データ量が増えると無駄な総当たり比較が増加してしまう。これに対して分割統治法（マージ・クイック）は、リストを二分し続けることで、全体の計算量が $O(N \log N)$ まで減らすことができる。より、データ量が膨大になればなるほど単純ソートのような無駄な総当たり比較が発生せず、計算回数の増大を防ぐことができるため、分割統治法は大量のデータを扱う上で有利であると考えられる。